AI SKILL SYSTEM · FOR CLAUDE CODE

# PDFjedi

A reusable framework for turning source PDFs into *complete* digital projects.

*Tech specs, UX/UI systems, contracts, budgets, and roadmaps — citation-grounded, production-ready, no invention.*

REPOSITORY
github.com/Gxnza48/PDFjedi

LICENSE
MIT

MAINTAINER
Gxnza48

CONTENTS

# What’s in this *document*

Eleven chapters covering the full PDFjedi system: what it produces, how it composes, how to drive it, and a worked example you can audit end-to-end.


01 **Introduction** → 03
What PDFjedi is, why it exists, and who it's for

02 **The output bundle** → 04
The nine artifacts produced by a full pipeline run

03 **Architecture** → 06
Three layers: master prompt, skills, templates

04 **The skills** → 07
Six composable modules — one role, one job each

05 **The templates** → 13
Structural contracts for each output artifact

06 **How to use** → 14
From clone to a signable bundle, step by step

07 **Workflow** → 16
The pipeline diagram and the dependencies it implies

08 **Example: Northbeam Coffee** → 17
A synthetic case study you can audit end-to-end

09 **Quality rules** → 19
The reasoning the master prompt enforces on every artifact

10 **Extending PDFjedi** → 20
Adding skills, templates, locales, and voices

11 **Credits & license** → 21
Author, libraries, license terms

01 · INTRODUCTION

# The most under-tooled step in client work, *turned into a framework.*

Reading a PDF and turning it into a project is the messiest step in real client delivery. The information is unstructured, contradictory, partly out of date, and almost never maps cleanly onto how an engineering team needs to receive it. PDFjedi treats that gap as the actual product surface.

---

## What PDFjedi *is*

PDFjedi is a modular AI skill system for Claude Code. It is a curated set of prompts, skills, and templates that compose together to convert messy real-world PDFs — briefs, RFPs, decks, research, meeting notes, compliance docs — into production-ready artifacts a team can execute on.

It is not a single prompt. It is a framework. Skills and templates can be loaded in part or in whole. You can run the entire pipeline (PDF in, full bundle out) or invoke a single skill (e.g. "rebuild only the budget after the client changed scope").

## Why it *exists*

Most AI tools that touch PDFs stop at a summary. A summary isn't a project. The team still has to translate it into requirements, then into a tech spec, then into a budget, then into a contract — losing fidelity at every hop. PDFjedi collapses those hops into one pipeline whose outputs are the artifacts the team would have built anyway, with every claim cited back to the source document.

## Who it's *for*

- **Agencies and freelancers** turning client briefs into proposals and statements of work.
- **Product teams** ingesting RFPs, vendor docs, or compliance PDFs.
- **Solo founders** converting investor decks or research PDFs into buildable plans.
- **Consultants** standardising how discovery documents become engagements.

## Design *principles*

01 **No invention.** If the PDF doesn't say it, PDFjedi doesn't claim it. Gaps go into an open-questions register, not into the artifacts.

02 **One skill, one job.** Skills are composable. A skill never silently triggers another — the master prompt orchestrates.

03 **Templates are contracts.** The output schema is fixed so downstream tooling can parse it and reviewers know where to look.

04 **Reasoning is visible.** Every artifact ends with a short "How this was derived" footer that lists which source sections drove which decisions.

05 **Senior-engineer voice.** Outputs read the way a senior engineer would write them — direct, decision-oriented, no filler.

| 9 | 6 | 4 | 1 |
|---|---|---|---|
| ARTIFACTS | SKILLS | TEMPLATES | PIPELINE · NO INSTALL |

02 · THE OUTPUT BUNDLE

# Nine artifacts. One folder. *Monday-morning* ready.

A full pipeline run emits one Markdown file per artifact into an `output/` folder. The two-digit prefixes are load-bearing — they encode the dependency order between steps.

---

```
output/
  ├── 01-requirements.md
  ├── 02-tech-spec.md
  ├── 03-ux-ui-system.md
  ├── 04-security-architecture.md
  ├── 05-roadmap.md
  ├── 06-budget.md
  ├── 07-proposal.md
  ├── 08-contract.md
  └── 99-open-questions.md
```

## What's in each *file*

01 · REQUIREMENTS

### Normalised requirements

Atomic claims extracted from every source PDF, deduplicated, normalised with stable REQ-IDs, MoSCoW-prioritised, and individually cited. Surfaces contradictions and gaps as separate tables.

02 · TECH SPEC

### Technical specification

Architecture, modules, data model, API surface, NFRs, operations. Every module references the REQ-IDs it satisfies; every REQ-ID is implemented or explicitly deferred.

03 · UX/UI

## UX/UI system

Screen inventory, component library, interaction flows, accessibility rules. Refuses to invent brand decisions — emits semantic tokens when none are supplied.

04 · SECURITY

## Security architecture

STRIDE threat model, data classification, identity & access, application and infrastructure controls, compliance mapping, incident response, launch-readiness checklist.

05 · ROADMAP

## Roadmap & milestones

Phases, parallel workstreams, dependencies, critical path. Cross-checked against the budget's PERT estimates so duration × team-size matches person-days.

06 · BUDGET

## Defensible budget

PERT-estimated line items, role mix, project management, QA, risk reserve, taxes, optional add-ons, out-of-scope. Refuses to invent rates.

07 · PROPOSAL

## Client-facing proposal

Plain-language deliverables, roadmap at a glance, team, investment, assumptions, next step. Written for a non-engineer decision-maker.

08 · CONTRACT

## Signature-ready contract

Jurisdiction-aware MSA. Deliverables mirror the proposal one-to-one. Standard clauses plus the locale-specific adjustments (IVA, GDPR, governing law). Lawyer review still required.

99 · OPEN QUESTIONS

## Open-questions register

Every gap, contradiction, and ambiguity surfaced as an explicit question rather than an invention. The register is non-empty by design — a zero count means the model hallucinated. Top three blocking items are flagged separately so the client knows what to answer first.

03 · ARCHITECTURE

# Three layers. Plain text. *No runtime.*

PDFjedi is prompts and contracts, not code. There is no executor, no daemon, no build step. The system is three layers of plain-text files that Claude Code loads into its working context.

---

L1

### Master prompt

`/prompts/master-prompt.md`

Defines the PDFJedi role, the operating contract (grounding, no-invention, templates-as-contract, reasoning-visible), the pipeline order, the reasoning rules for ambiguous source material, and the quality gates every artifact must pass before emission. Always loaded first; always in effect.

L2

### Skills (modular)

`/skills/*.md`

Each skill is one role, with strict inputs, a numbered process, an output schema, and a set of quality checks. Skills don't invoke each other — composition happens at the master-prompt layer. New skills slot in by following the same file structure.

L3

### Templates (structural contracts)

`/templates/*.md`

Define the exact section structure of every output artifact: ordered numbered sections, fixed table column headers, bracketed placeholders for fields the skill will fill, and the "How this was derived" footer. Templates contain no instructions — that lives in skills.

## Why three *layers*

Concerns are separated so each layer can change independently:

- **Master prompt** changes when the philosophy of PDFjedi changes (rules, voice, what's never done). Rare.
- **Skills** change when a process changes (how to extract requirements, how to estimate a budget). Occasional.
- **Templates** change when an output schema changes (a new section in the contract, a new field in the budget). Frequent — driven by client feedback.

Keeping templates separate from skills means you can rebrand or relocalise PDFjedi by editing only templates. The skills still work. Adding a new artifact type means dropping a skill plus a template, then adding one line to the master prompt's pipeline table. No code changes anywhere.

> **Why no code?**
>
> PDFjedi could be implemented as a CLI tool that parses PDFs and calls an LLM. That would be brittle (PDF parsing alone is a project) and would lock the framework to one provider. Keeping it as prompts plus skills plus templates means it runs anywhere Claude Code runs, any team member can read and modify a skill without programming, and the bundle is forkable per-client or per-engagement without code review.

# Six skills. *Composable, not coupled.*

Each skill is one role with one job. They follow the same internal structure — Role, Inputs, Process, Output, Quality checks, Anti-patterns — so they can be composed without knowing each other's internals.

| SKILL | CONSUMES | PRODUCES | HEADLINE GUARANTEE |
|---|---|---|---|
| **pdf-analysis** | Source PDFs | 01-requirements.md | Every claim downstream is traceable to a citation in source. |
| **tech-spec-builder** | 01-requirements.md | 02-tech-spec.md | Every REQ-ID is implemented by a module or explicitly deferred. |
| **ux-ui-designer** | 01 + 02 | 03-ux-ui-system.md | Every user-facing REQ-ID maps to at least one screen. |
| **security-architect** | 01 + 02 + 03 | 04-security-architecture.md | Every PII field is addressed by a control. |
| **budget-generator** | 02 + 05-roadmap | 06-budget.md | Every line item references a REQ-ID; PM, QA, and a risk reserve are present. |
| **contract-generator** | 06 + 07 + 05 | 08-contract.md | Deliverables match the proposal one-to-one; totals match the budget. |

**The skill template**

Every skill file is structured the same way: *Role* (what it pretends to be), *Inputs* (which prior artifacts it consumes), *Process* (numbered steps), *Output* (the exact schema, plus a "How this was derived" footer), *Quality checks* (a self-audit before emission), and *Anti-patterns* (what to refuse).

04.1 · SKILL PROFILE

# pdf-analysis

Extract structured, grounded data from one or more PDFs. Merge them. Detect contradictions. Emit the canonical requirements file that every downstream skill consumes.

---

/SKILLS/PDF-ANALYSIS.MD  FOUNDATION SKILL  OUTPUT · 01-REQUIREMENTS.MD

## Role

A domain-agnostic requirements analyst. Reads PDFs the way a senior engineer reads them on day one: sceptically, looking for what's said, what's almost said, what's contradicted, and what's missing. Does not summarise — extracts and normalises.

## Process

01 **Inventory.** For each PDF: filename, page count, type, date (flag if missing), author, one-line role.

02 **Extract atomic claims.** One fact, requirement, constraint, decision, or quote per claim. Each tagged and cited.

03 **Normalise.** Assign stable REQ-IDs. Merge near-duplicates across documents. Convert vague terms ("fast", "scalable") into measurable form — or into an open question.

04 **Detect contradictions.** Pairs of claims that cannot both be true. Recorded with both citations; never resolved by PDFjedi.

05 **Detect gaps.** Anywhere a senior engineer would need more information to implement.

06 **Classify priority.** MoSCoW, driven by the source language. Never silently upgrade a Should to a Must.

## Quality checks

- Every claim has a citation.
- No requirement contains "fast", "scalable", "modern", "user-friendly", or "seamless" without an open question pointing at it.
- At least one open question — if there are zero, the model almost certainly missed real gaps and the analyst re-checks.

- Every requirement ID is unique.

## Anti-patterns it refuses

- Producing a prose summary instead of structured tables.
- Picking a winner when two PDFs disagree.
- Inferring requirements from "industry norms" — if the PDF didn't say it, it doesn't exist.
- Silently upgrading priority because something "feels critical".

04.2 · SKILL PROFILE

# tech-spec-builder

Convert normalised requirements into an unambiguous technical specification an engineer can execute against without re-reading the source PDFs.

---

/SKILLS/TECH-SPEC-BUILDER.MD  OUTPUT · 02-TECH-SPEC.MD

TEMPLATE · TECH-SPEC-TEMPLATE.MD

## Role

A staff-level engineer scoping a new project. Translates requirements into architecture, modules, data, APIs, and operations decisions. Makes conservative, defensible technology choices and documents the reasoning.

## The boring-tech rule

When the source doesn't pin the stack, the skill picks the most-hireable, most-supported credible option for each tier. Exotic choices have to be justified in writing or they aren't made.

| TIER | DEFAULT STARTING POINT |
|---|---|
| Frontend | Next.js + TypeScript + Tailwind |
| Backend | Node.js (Fastify / NestJS) or Python (FastAPI) — pick by team skill |
| Database | PostgreSQL |
| Background jobs | Managed queue or Redis-backed job runner |
| Auth | Hosted (Auth0 / Clerk / Cognito) unless self-hosted is required |
| Hosting | Major cloud or a PaaS (Vercel + Supabase, Render + Neon) |

## Disambiguation strategy

When the requirements are vague, the skill picks *the most specific defensible decision* and flags the ambiguity. "Must be fast" becomes "P95 < 2s on broadband" plus an open question. "Scalable" becomes "10× current load without rewrite" plus an open question asking for current load.

## Quality checks

- Every module maps to ≥ 1 REQ-ID; every REQ-ID is satisfied or deferred.
- Every integration lists protocol, auth, and direction of data flow.
- Data model lists PII fields explicitly.
- NFRs either have a number or an open question — never wishful adjectives.
- Stack choices each have a one-to-two-line rationale.

04.3 · SKILL PROFILE

# ux-ui-designer

Produce a complete UX/UI system: screen inventory, information architecture, components, interaction patterns, accessibility — derived strictly from the requirements. Never invents a brand.

---

/SKILLS/UX-UI-DESIGNER.MD  OUTPUT · 03-UX-UI-SYSTEM.MD  DEFAULT TARGET · WCAG 2.1 AA

## Role

A senior product designer working alongside engineering. Thinks in flows and states, not pixels. Produces the written design system that engineering and design will then realise visually.

## What it produces

- **Users & jobs** — one row per role, with top jobs-to-be-done and linked REQ-IDs.
- **Information architecture** — a site map with auth gating per page.
- **Screen inventory** — each screen has purpose, inputs (data fetched), outputs (user actions), states (empty / loading / success / error), and linked REQ-IDs.
- **Component inventory** — atoms, molecules, organisms, patterns; each with variants, props, and where used.
- **Interaction patterns** — auth, primary CRUD, payment, search, settings. Each flow has an explicit error path.
- **Visual system** — type scale, semantic colour tokens, spacing, radius. *Tokens only* when no brand is supplied.
- **Accessibility** — WCAG target, contrast, keyboard, focus, ARIA per pattern.
- **Responsive strategy** — breakpoints, mobile-first or not, components that change behaviour at breakpoints.
- **Content & copy rules** — voice, microcopy, number/date/currency formatting, translation strategy.

## What it refuses

- Inventing colours, fonts, logos, or brand decisions absent from the source.
- Listing screens without states.
- Skipping accessibility because the brief didn't mention it.

- Producing prose paragraphs instead of structured inventories.

04.4 · SKILL PROFILE

# security-architect

Define a secure-by-design architecture: threat model, controls, data-handling rules, identity and access, compliance mapping, and checklists the team can verify against.

---

/SKILLS/SECURITY-ARCHITECT.MD  OUTPUT · 04-SECURITY-ARCHITECTURE.MD

BASELINE · OWASP ASVS L2

## Sections produced

01 **Scope & compliance regimes** — only what the source brings into scope. GDPR, HIPAA, SOC 2, PCI-DSS, Ley 25.326, LGPD.

02 **Data classification** — every tech-spec entity is mapped to a class: Public, Internal, Confidential, PII, Sensitive PII, Auth Secret.

03 **Threat model (STRIDE)** — 10–20 real threats per major component, with mitigation and residual risk. Not an exhaustive catalogue.

04 **Identity & access** — auth method, sessions, RBAC matrix, service-to-service auth, account lifecycle.

05 **Data protection** — TLS, encryption at rest, backups, log PII allow-list, residency.

06 **Application security controls** — input validation, output encoding, CSRF, rate limiting, security headers, secrets, audit trail.

07 **Infrastructure security** — network segmentation, IAM, image scanning, DR posture.

08 **Compliance mapping** — each regulation article mapped to the control that addresses it.

09 **Incident response** — detection, triage roles, containment, notification, post-mortem.

10 **Launch-readiness checklist** — every item testable, every item owned.

## Calibration

The skill matches rigour to risk. An internal tool with no PII gets a light version. Customer-facing SaaS with PII gets all sections, full STRIDE, full compliance mapping. Regulated industries (health, finance) get all of the above plus a written recommendation that a specialist auditor be engaged — recorded as an open question.

04.5 · SKILL PROFILE

# budget-generator

Produce a structured, defensible budget with line items, role mix, assumptions, taxes, and a risk reserve. Aligned line-for-line with the tech spec and the roadmap.

---

/SKILLS/BUDGET-GENERATOR.MD  OUTPUT · 06-BUDGET.MD  ESTIMATION · PERT (O · L · P)

## Estimation

Effort is estimated per line item using the three-point PERT method:

```
expected = (optimistic + 4 × likely + pessimistic) / 6
```

Person-days are rounded to half-days. Total expected effort, divided by team size, must roughly equal the roadmap duration — if the two are off by more than 30%, one of them is wrong and the budget halts until it's reconciled.

## Mandatory line items

- **Contingency / risk reserve** — default 15%, raised to 20–25% for discovery-heavy or vague scopes, lowered to 10% only when the spec is unusually tight.
- **Project management** — a real line, not buried in overhead. Typically 10–15% of engineering effort.
- **QA & UAT** — separate; not a free side-effect of engineering.
- **DevOps / environments** — set-up and ongoing.
- **Discovery / kickoff** — if the project starts with discovery, it's priced.
- **Warranty period** — typically 30 days of free bug-fix capacity post-acceptance, called out as zero-cost so the client knows it's included.

## What it refuses

- A single lump-sum number with no breakdown.
- Hiding PM in "overhead".
- A budget without a risk reserve.
- Inventing rates to make the math work. If rates are unsupplied, they remain bracketed placeholders and become open questions.

## The discovery-quote escape valve

When the source PDF is too vague to defend a number, the skill produces a small fixed-price discovery quote (one to two weeks) instead of a full project quote — plus a clearly-labelled rough-order-of-magnitude range. More honest than the alternative.

04.6 · SKILL PROFILE

# contract-generator

Draft a professional, signature-ready service contract that reflects the proposal exactly and complies with the locale and jurisdiction provided. Not legal advice — a high-quality first draft a lawyer can review in 30 minutes instead of three hours.

---

/SKILLS/CONTRACT-GENERATOR.MD  OUTPUT · 08-CONTRACT.MD  NOT LEGAL ADVICE

## Variants by pricing model

- **Fixed-price** — milestone-based payment, bounded scope, mandatory change-order process.
- **Time-and-materials** — rate card plus cap, invoicing cadence, time-reporting clause.
- **Retainer** — recurring fee, included capacity, scope ceiling, rollover policy.

## Mandatory clauses

Every variant includes, at minimum: scope of work, change-order process, acceptance (default five-business-day silent acceptance), intellectual property, confidentiality, data protection, warranty, limitation of liability (capped at 12 months of fees), termination (for cause, for convenience, kill-fee), force majeure, governing law & jurisdiction, notices, entire agreement, signatures.

## Jurisdiction-aware adjustments

| LOCALE | ADJUSTMENT |
|---|---|
| Argentina | Spanish or bilingual; IVA where applicable; Tribunales Ordinarios de la Ciudad Autónoma de Buenos Aires unless specified. |
| Spain / EU | GDPR-compliant data clauses; Reglamento (UE) 2016/679. |
| USA (Delaware) | English; arbitration clause acceptable; reference to Delaware General Corporation Law where relevant. |
| UK | English; UK GDPR + DPA 2018; Limitation Act 1980 for warranty periods. |
| LATAM (general) | Local currency or USD with conversion clause; local tax wrapper. |

## What it refuses

- Copying generic US boilerplate into a non-US contract.
- Setting unlimited liability or open-ended indemnity.
- Omitting a change-order process.
- Using vague deliverables — every deliverable has objective acceptance criteria.
- Claiming to be final legal text. Lawyer review is recommended every time.

05 · THE TEMPLATES

# Four templates. *Structural contracts* for every output.

Templates define the exact section structure of each artifact: ordered numbered sections, fixed table column headers, bracketed placeholders for fields the skill will fill, and the "How this was derived" footer. They contain no instructions — that lives in skills.

---

/TEMPLATES/TECH-SPEC-TEMPLATE.MD

## Tech spec template

11 numbered sections: Overview, Users & roles, Functional scope (by domain), Architecture (diagram, components, integrations), Data model, API surface, Non-functional requirements, Operations, Tech-stack decisions, Risks & unknowns, Out of scope.

/TEMPLATES/CONTRACT-TEMPLATE.MD

## Contract template

Full MSA: 18 numbered sections (Scope, Term, Fees, Independent contractor, Acceptance, Change orders, IP, Confidentiality, Data protection, Warranty, LoL, Indemnification, Termination, Force majeure, Disputes, Notices, Miscellaneous, Governing law) plus Exhibits A–E.

/TEMPLATES/BUDGET-TEMPLATE.MD

## Budget template

Summary table, rate card, phase-grouped line items with PERT estimates, project-wide lines (PM, QA, discovery, DevOps, warranty), payment schedule (per pricing model), optional add-ons, out-of-scope list, assumptions, client notes.

/TEMPLATES/PROPOSAL-TEMPLATE.MD

## Proposal template

Non-engineer-facing: Context, Deliverables table, Approach narrative, Roadmap at a glance, Team, Investment, Assumptions, Why us, What we need from you, Next step, Appendix.

> **Templates are the output contract**
> When a skill emits an artifact, it follows the template exactly. Sections cannot be reshaped or dropped. If a section does not apply to a given project, the skill writes "Not applicable." plus one sentence explaining why — never silence.

## Localising or rebranding

Because templates are pure structure, you can localise (e.g. ship a Spanish-language contract template) or rebrand (e.g. ship a corporate-tone proposal template) by editing only the template file. The skill that fills it is unaffected. Fork the template, change the master prompt to load the forked version, and the pipeline picks it up.

06 · HOW TO USE

# Three steps from PDF to *output bundle.*

There is nothing to install. PDFjedi is loaded as context into Claude Code, then driven with a single natural-language instruction. The framework runs inside the model session — your repo is the source of truth, not an executor.

---

## Quick start — full pipeline

01 **Open Claude Code in this repo.**

```
$ git clone https://github.com/Gxnza48/PDFjedi.git
$ cd PDFjedi
$ claude
```

02 **Load the framework.** Tell Claude to load the master prompt, every skill, and every template. From here on it is PDFJedi.

```
# Load:
/prompts/master-prompt.md
/skills/*
/templates/*
```

03 **Attach the PDF(s) and ask.**

> *"Analyze this PDF and generate a full project:*
> *tech spec, UX/UI system, contract, budget, roadmap."*

## Targeted use — single skill

Each skill is self-contained. To rebuild only the budget after the client changed scope, load:

```
# Load:
/prompts/master-prompt.md
/skills/budget-generator.md
/templates/budget-template.md
```

Then issue an instruction like "Build a budget for the project described in the attached tech spec. Currency USD. Pricing model fixed-price. Risk reserve 15%."

## Composed use — multi-document

Throwing multiple PDFs in together produces a better result than running them one-by-one, because contradictions get surfaced. Load `pdf-analysis.md` plus `tech-spec-builder.md`, attach the brief plus the meeting notes plus the security requirements, and ask Claude to merge them and flag contradictions before building the tech spec.

# Editing after a *run.*

Two reliable patterns for updating an existing output bundle. Both keep the bundle internally consistent.

---

## Pattern 1 — re-run with an answer

If the open-questions register had items and you now have answers:

> *"Here are answers to the open questions:*
> *- OQ-001: Performance target is P95 < 1.5s.*
> *- OQ-003: Team will be 1 senior + 1 mid, no dedicated QA.*
> *- OQ-007: GDPR is in scope; LGPD is not.*
>
> *Re-run the pipeline incorporating these answers.*
> *Update only the artifacts that change."*

PDFjedi identifies which artifacts are affected by the change (via the dependency graph) and rewrites only those.

## Pattern 2 — targeted change

If the client changed one thing:

> *"The client moved the launch from 2026-09-01 to*
> *2026-10-15 and added a Spanish locale.*
> *Update the roadmap, budget, proposal, and contract.*
> *Surface any new open questions."*

## The recommended review order

Once you have a bundle, read it in this order — it's the fastest path to a clean version:

01 **99-open-questions.md** — answer or accept each item before sharing externally.

02 **01-requirements.md** — verify nothing important is missing or invented.

03 **02-tech-spec.md** + **04-security-architecture.md** — engineering sign-off.

04 **06-budget.md** — commercial sign-off.

05 **07-proposal.md** + **08-contract.md** — share with legal and the client only after the above are clean.

> **Modes**
> The pipeline runs in `full`, `triage` (just requirements + open questions, fast read), `quote-only` (skip UX/UI and security; useful when the client only wants a number), or `single-artifact: <name>`. Mode is set via the Parameters block in `/prompts/pdf-to-product.md`.

07 · WORKFLOW

# The pipeline, the dependencies, *the failure modes.*

Each step consumes specific upstream artifacts and produces exactly one downstream artifact. The dependency graph is what makes minimal re-runs possible — when an input changes, only the affected steps rerun.

## Dependencies

| # | SKILL | CONSUMES | PRODUCES |
|---|---|---|---|
| 1 | pdf-analysis | Source PDF(s) + in-chat context | 01-requirements.md |
| 2 | tech-spec-builder | 01-requirements.md | 02-tech-spec.md |
| 3 | ux-ui-designer | 01 + 02 | 03-ux-ui-system.md |
| 4 | security-architect | 01 + 02 + 03 | 04-security-architecture.md |
| 5 | (inline) roadmap | All upstream | 05-roadmap.md |
| 6 | budget-generator | 02 + 05 | 06-budget.md |
| 7 | (inline) proposal | 01 + 05 + 06 | 07-proposal.md |
| 8 | contract-generator | 05 + 06 + 07 | 08-contract.md |
| 9 | (inline) open-questions | All steps | 99-open-questions.md |

## Failure modes & recovery

| FAILURE | HOW IT MANIFESTS | RECOVERY |
|---|---|---|
| Source PDF unreadable | Step 1 halts with an explanatory message | Re-OCR / re-export the PDF |
| Contradiction blocks pipeline | Step 1 emits requirements + open questions and refuses to proceed | Resolve top contradiction, re-run |
| Vague brief, empty NFRs | Tech spec emits open questions instead of numbers | Provide NFR targets in chat, re-run from step 2 |
| Budget can't be defended | budget-generator emits a discovery quote instead | Accept the discovery quote or supply more inputs |
| Jurisdiction unknown | contract-generator stops at §3 | Set the locale parameter, re-run from step 8 |

In every case, the pipeline degrades gracefully — it produces less, not wrong.

08 · EXAMPLE

# Northbeam Coffee. *A worked case* end-to-end.

The repo ships with a synthetic Argentine specialty-coffee project under `examples/`. Two source documents — a client brief and an undated follow-up call — designed to contradict each other in three places so the pipeline's open-questions discipline is visible.

---

## Input

EXAMPLES/INPUT/BRIEF.PDF

### Client brief — 4 pages

From Maya Rios, Head of Operations. Describes Northbeam's wholesale ordering pain at 40 accounts (target 200 by EOY), requested features, budget (~USD 50,000), launch (ideally end of August), AWS hosting, Spanish UI, GDPR for the Spain customer.

EXAMPLES/INPUT/MEETING-NOTES.PDF

### Follow-up call — 1 page, undated

From a call with Maya and CFO Diego. Hardens the budget to USD 40,000 hard ceiling, hardens the launch to 2026-09-01, upgrades English from "later" to "in scope at launch", forbids US data residency.

> **Three intentional contradictions**
> CTR-001 budget (50k vs 40k); CTR-002 launch date ("end of August" vs hard 2026-09-01); CTR-003 English locale ("not for v1" vs "in scope at launch"). The two PDFs together test whether the pipeline picks a winner — it doesn't.

## What the pipeline produced

| ARTIFACT | HEADLINE NUMBERS |
| --- | --- |
| 01-requirements.md | 21 normalised requirements, 47 atomic claims, 3 contradictions, 8 open questions |
| 02-tech-spec.md | 7 functional domains, Next.js + Postgres + AWS eu-west-1, region decision documented |
| 03-ux-ui-system.md | 19 screens, 17 components, WCAG 2.1 AA, semantic tokens (no brand supplied) |
| 04-security-architecture.md | STRIDE on 12 components, GDPR + Ley 25.326 mapping, 17-item launch checklist |
| 05-roadmap.md | 8 phases over 18 weeks, hard launch 2026-09-01, critical path through reorder UX |
| 06-budget.md | Two variants (USD 46k recommended / USD 38k 40k-ceiling), 119.6 person-days, 18% reserve |
| 07-proposal.md | 10 plain-language deliverables, milestone-based payments |
| 08-contract.md | Argentina-governed MSA, Tribunales CABA, IVA-aware, Spanish-translation clause |
| 99-open-questions.md | 11 OQs · Top 3 blocking: precedence between PDFs, brand asset handover, Tango integration |

# What to look for when you *audit it.*

The example is most useful as a reading exercise. These are the moments where PDFjedi's discipline shows up most clearly — read with these in mind.

---

## Contradictions surfaced, never resolved

Open `01-requirements.md` §Contradictions detected. Three rows, each with both citations. PDFjedi does not pick the more recent document, does not pick the more conservative number, does not pick anything — it asks the client (OQ-001).

## Vague language disambiguated

"Modern and professional" (brief, p.2) becomes *"component-driven, design-system based, accessible to WCAG 2.1 AA"* with a corresponding open question (OQ-002) asking for brand assets. The phrase never appears verbatim in the tech spec.

## Brand decisions deferred, not invented

The UX/UI system uses semantic tokens — `color.brand.primary`, `font.body` — with TBD values pending Figma access. Compare with the typical "AI generates a teal-and-purple visual identity" output from less disciplined tools.

## Region decision shown, not buried

REQ-017 says Argentina or EU, not US. There is no AWS region in Argentina; `sa-east-1` is São Paulo. PDFjedi picks `eu-west-1` to satisfy both the rule and the Spain-customer expectations — and writes the reasoning into the tech spec's stack-decisions table. The alternative is flagged as a one-line config change in OQ-010.

## Two budget variants, not one

Because of CTR-001 (budget conflict), `06-budget.md` presents both **Variant A — Full v1** (USD 46k recommended) and **Variant B — 40k ceiling** with explicit scope reductions (defer EN locale, reduce admin pages, narrow a11y scope). The client picks at signature.

## Contract is jurisdiction-aware

The contract is governed by Argentine law, dispute resolution at Tribunales Ordinarios de la Ciudad Autónoma de Buenos Aires, IVA 21% treatment per §3.2, language clause noting the Spanish translation is for convenience only, and a comment block flagging that the jurisdiction city and the English-vs-Spanish controlling-language choice both require lawyer review.

09 · QUALITY RULES

# The reasoning the master prompt *enforces.*

PDFjedi is opinionated about how to decide things when the source is ambiguous. These rules live in the master prompt and apply to every artifact.

---

## Disambiguate before deciding

If the source uses a fuzzy term ("scalable", "modern", "secure", "user-friendly"), PDFjedi picks *the strongest concrete reading the source supports* and records the ambiguity as an open question. Never silently chooses a meaning.

## Cheapest-credible interpretation

When a requirement could be read two ways, PDFjedi picks the reading with the lowest credible delivery cost — and flags it. The client can upgrade scope; they can't unwind unflagged over-engineering.

## Contradictions are findings, not problems

If two PDFs contradict each other, both versions go into open questions verbatim with citations. The client is the tiebreaker.

## Boring tech wins

When the source doesn't specify a stack, prefer the most boring credible option (well-known, hireable, supported). Justify exotic choices in writing or don't make them.

## Round-trip test

Before emitting any artifact, PDFjedi asks: *if a senior engineer read only this artifact, could they execute the work without re-reading the source PDF?* If no, the artifact is incomplete — it's fixed before emission, not surfaced as a caveat.

## The Monday-morning test

The full bundle must let a small team start work on Monday: scope is bounded, dependencies are mapped, environment is buildable, the first ticket is obvious. If the bundle fails this test, the run isn't done.

## Quality gates run before every emission

- Every numeric claim has a citation.
- Every requirement has a unique ID referenced from the tech spec, UX system, and budget line items.
- No artifact contradicts another (e.g. budget includes a module the tech spec doesn't define).
- No artifact contains TODOs, template placeholders, or "[REPLACE]" markers.
- The open-questions register is non-empty (a zero count almost always means hallucination).
- The roadmap's critical path is consistent with the budget's phasing.
- The contract's deliverables match the proposal's deliverables one-to-one.

# Add a skill in *three files.*

**Because PDFjedi is prompts and contracts rather than code, extension is just dropping new Markdown files in the right folders and adding a line to the master prompt's pipeline table.**

---

## To add a new artifact type

01 Create `templates/<artifact>-template.md` with the section structure.

02 Create `skills/<skill>.md` following the standard structure: Role, Inputs, Process, Output, Quality checks, Anti-patterns.

03 Add a row to the pipeline table in `/prompts/master-prompt.md` §4.

No code changes anywhere. Optionally add a runbook variation in `/prompts/pdf-to-product.md` so the new step is part of the canonical full run.

## To localise for a new jurisdiction

Edit `templates/contract-template.md` to add jurisdiction-specific clauses, or fork it as `contract-template.<locale>.md`. Add the new locale to the table in `skills/contract-generator.md`. Update the Locale parameter examples in `/prompts/pdf-to-product.md`.

## To change voice or branding

Edit the *"Tone for client-facing artifacts"* section in the master prompt, or fork the entire master prompt as `master-prompt.<variant>.md` and load it in place of the default. Same skills, different voice.

## To swap the default stack

Edit the *"Default starting point"* block in `skills/tech-spec-builder.md`. The boring-tech rule is implemented as a table — change the table, change the defaults. No other file is affected.

## Composition rules

- Skills are leaves, not internal nodes. A skill never invokes another skill — the master prompt orchestrates.

- Templates are structural, not behavioural. Don't put instructions inside templates; they'll be filled into the output.
- One file, one responsibility. If a file does two things, split it.
- Idempotent re-runs. A skill given the same inputs produces the same output (modulo the model's natural variance).

# Credits & *license.*

PDFjedi is open-source. The skills, templates, and prompts are released under the MIT license. Fork it, customise it per client, ship work with it.

---

MAINTAINER

### Gxnza48

github.com/Gxnza48 · Author of the PDFjedi repository and the accompanying landing page.

RUNTIME

### Claude Code

Anthropic's CLI / IDE / web client for Claude. PDFjedi is loaded as context into a Claude Code session.

LANDING PAGE

### GSAP + Lenis

GreenSock Animation Platform for animations and scroll-driven reveals. Lenis for smooth scrolling.

ICONOGRAPHY

### Bootstrap Icons + Devicons

Bootstrap Icons for UI iconography. Devicons for the stack-agnostic technology row on the landing page.

TYPOGRAPHY

### Space Grotesk · Inter · Instrument Serif · JetBrains Mono

Google Fonts. Display, body, italic accents, and code respectively — same family across the landing page and this document.

CLAUDE CODE ICON

### lobe-icons

github.com/lobehub/lobe-icons. Open-source icon set; the Claude Code mark used in the landing page and this document is taken from there.

## License

The PDFjedi repository is released under the **MIT License**. You can use it, fork it, customise it, ship client work with it, and redistribute it. The only requirements are that you keep the license notice intact and don't claim that this software comes with any warranty — it doesn't.

## Not legal advice

The contract artifacts PDFjedi produces are professional drafts. They are *not legal advice*. A licensed attorney in the governing jurisdiction must review any contract before either party signs. The same applies to the security architecture: launch-readiness checklists are not a substitute for a real pen test.

---